



Gradle
From "It Works" to "I Understand It"
André Martins

WHO_AM_I

- Bsc Computer Science and Engineering @ **ISEL**
- Msc Computer Science and Engineering @ **IST**
- Backend Software Engineer @ **Sky Portugal**
- Community Builder @ **Tech na Calçada**
- **JNation Champion**
- Tutor / Teacher as Freelancer
- Juggler



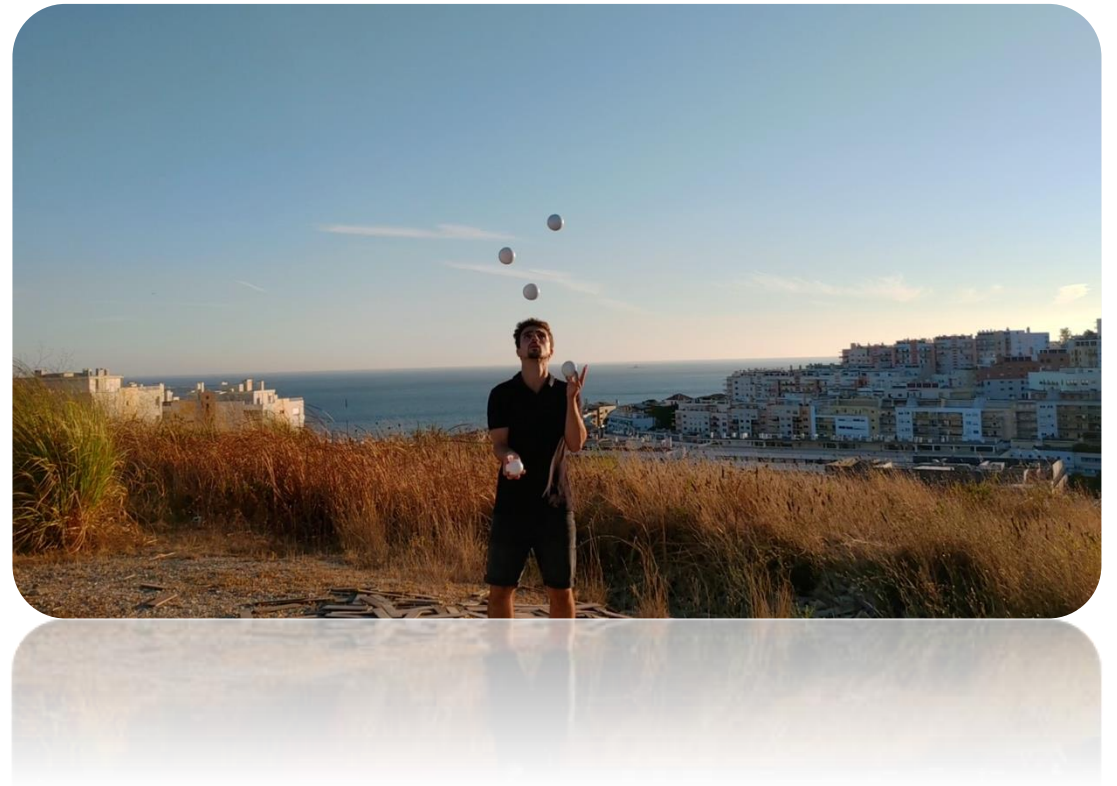
chriptus13



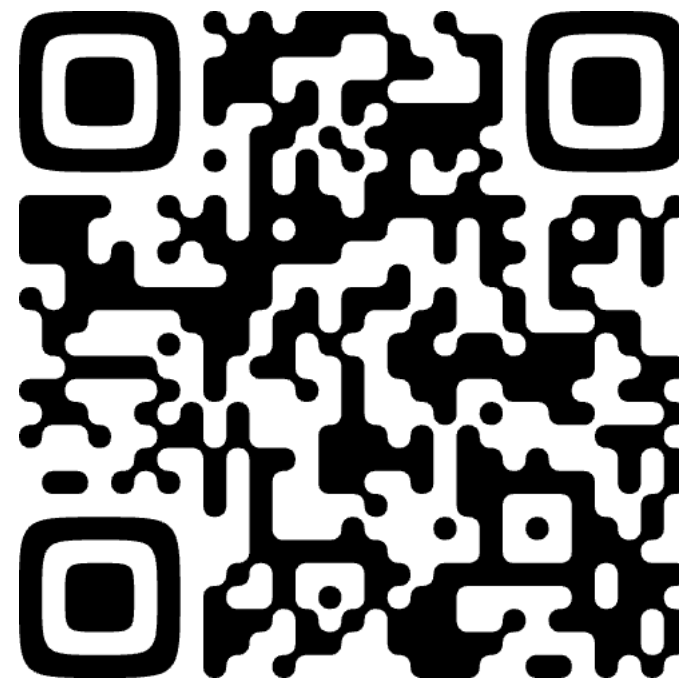
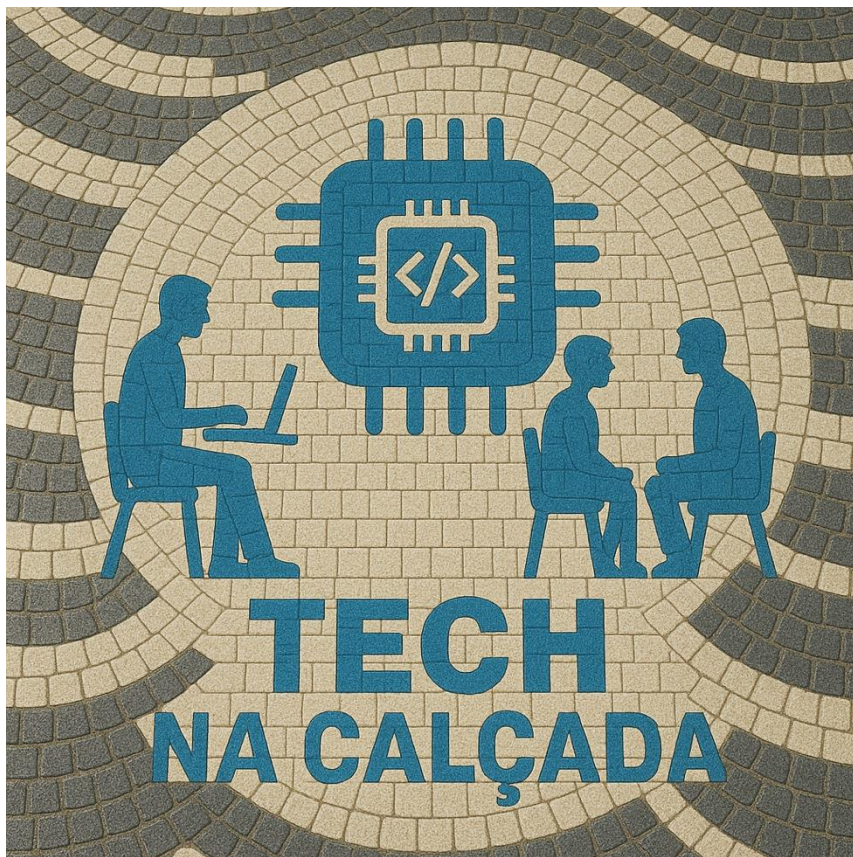
/in/andremartins13



andre@andremartins.pt



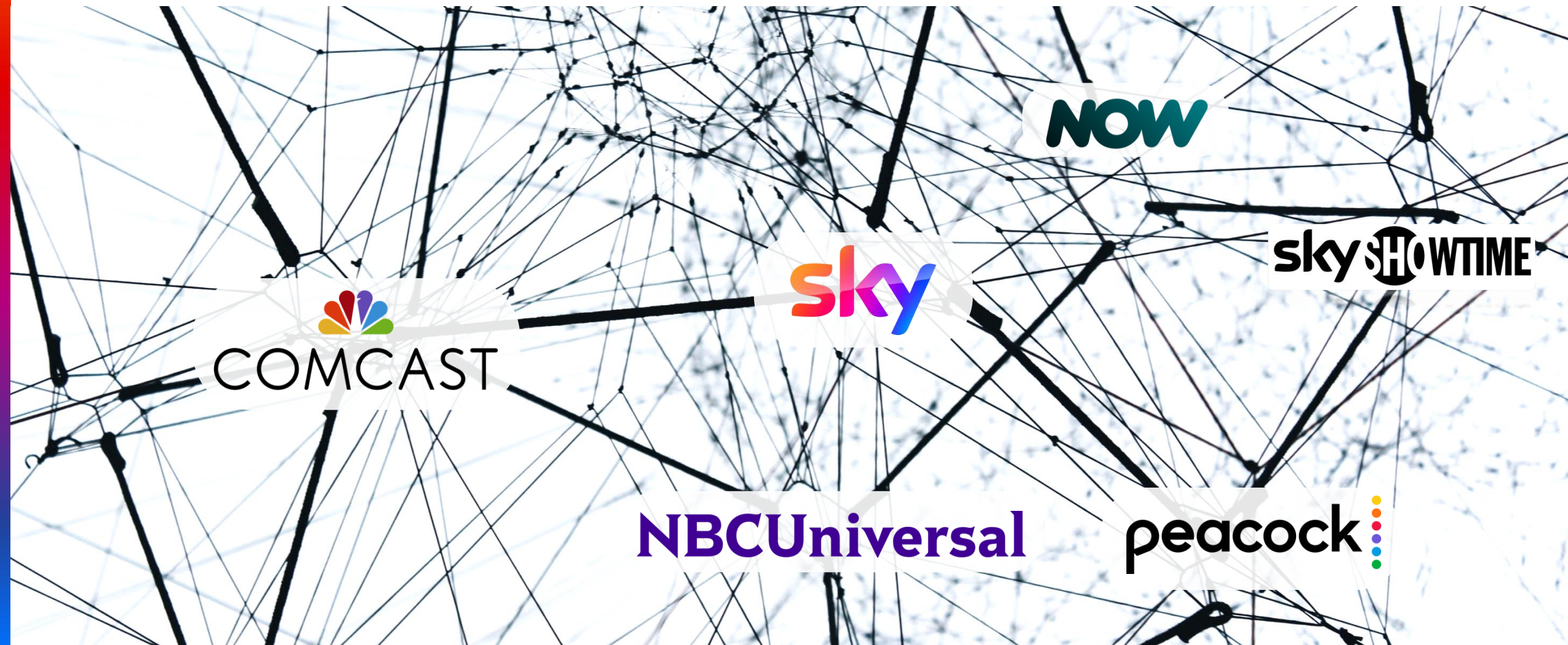
Tech na Calçada



lu.ma/tech-na-calcada

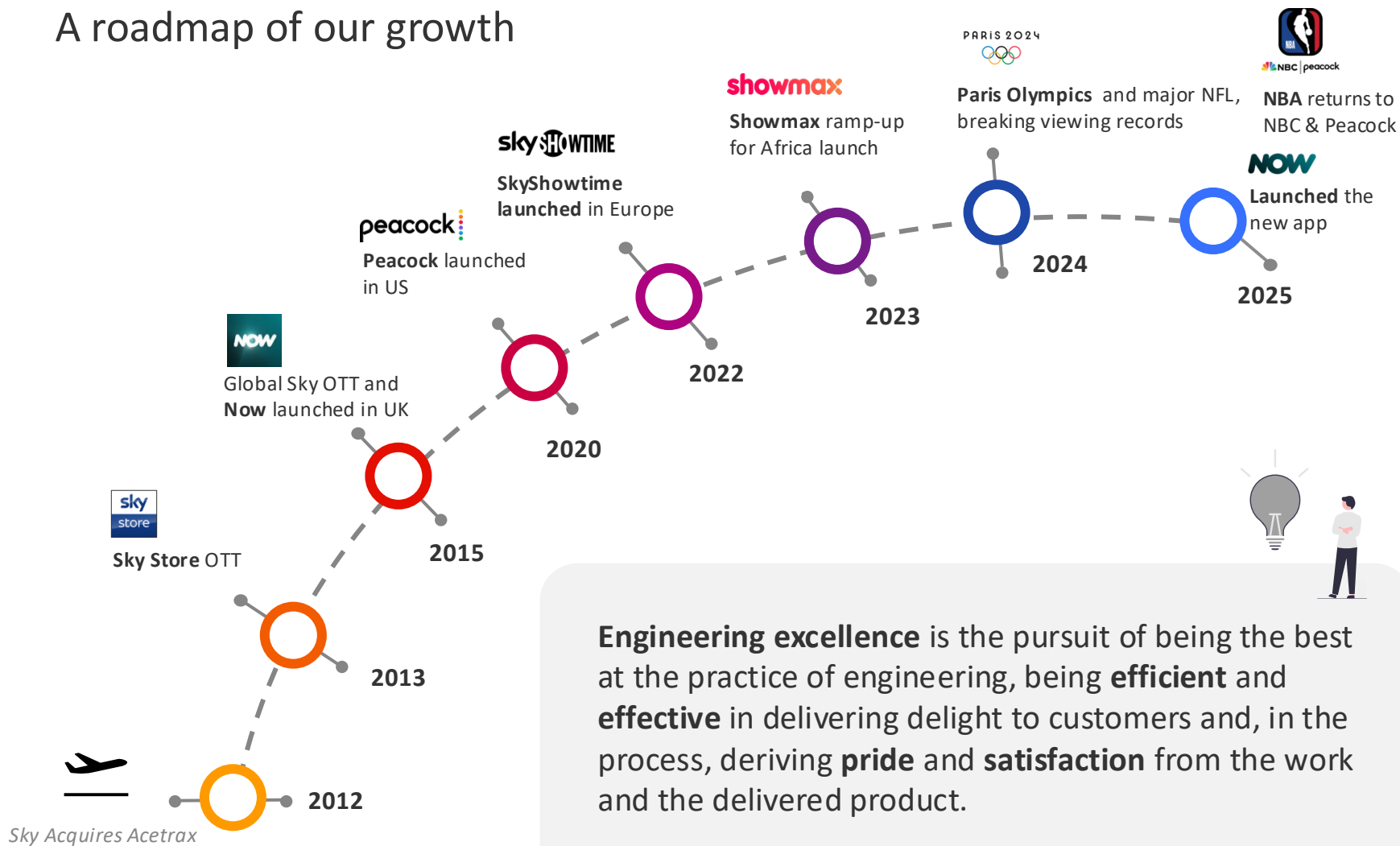
Engineering Excellence at Scale

Powering global platforms for millions of viewers across continents, from Peacock, NOW, and SkyShowtime



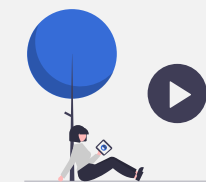
Sky Portugal Journey

A roadmap of our growth



Engineering excellence is the pursuit of being the best at the practice of engineering, being **efficient** and **effective** in delivering delight to customers and, in the process, deriving **pride** and **satisfaction** from the work and the delivered product.

Adapted from <https://medium.com/swlh/what-is-engineering-excellence-a8aa5a1e8dc5>



Engineering Principles

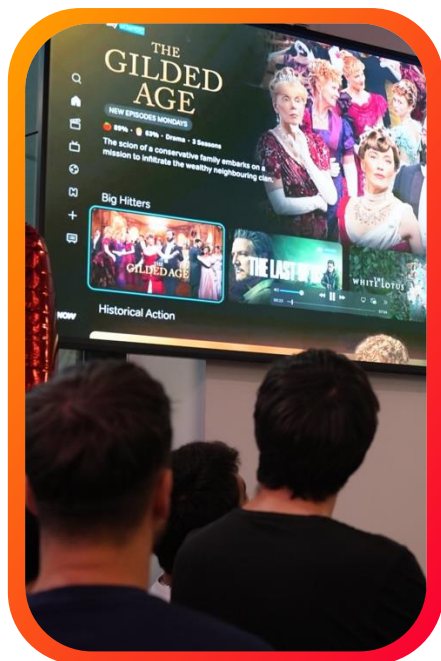
- **Multi-everything:** Build flexible and adaptable solutions. Build capabilities instead of features.
- **Hands-on & can do attitude:** Focus on the solutions and how to move forward.
- **Customer focus:** The user is King, whatever makes the experience more enjoyable and resilient.
- **Built it, own it:** Be responsible and take pride in what you build



Years of Touchdowns and Home Runs

Looking at key milestones and achievements from the last few months

Project
GOLD



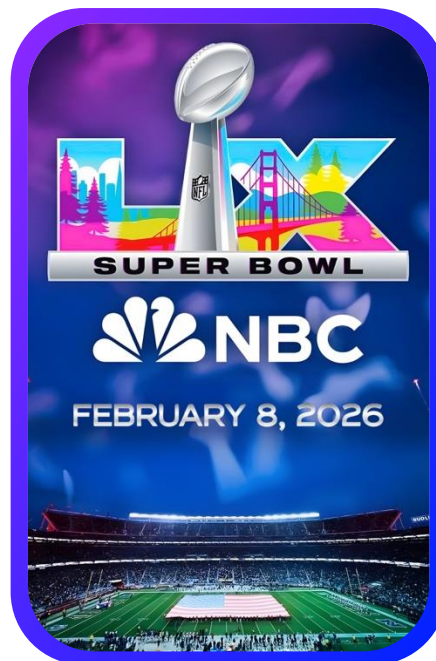
NBA Debut on
Peacock



Milan Cortina
Winter Olympics



Super Bowl
LX



NBA
All Star Game



Legendary February

Agenda

1. Build Lifecycle
2. Version Catalogs
3. BuildSrc vs Composite Builds
4. Toolchains
5. Tips & Tricks
6. Demo
7. Q&A





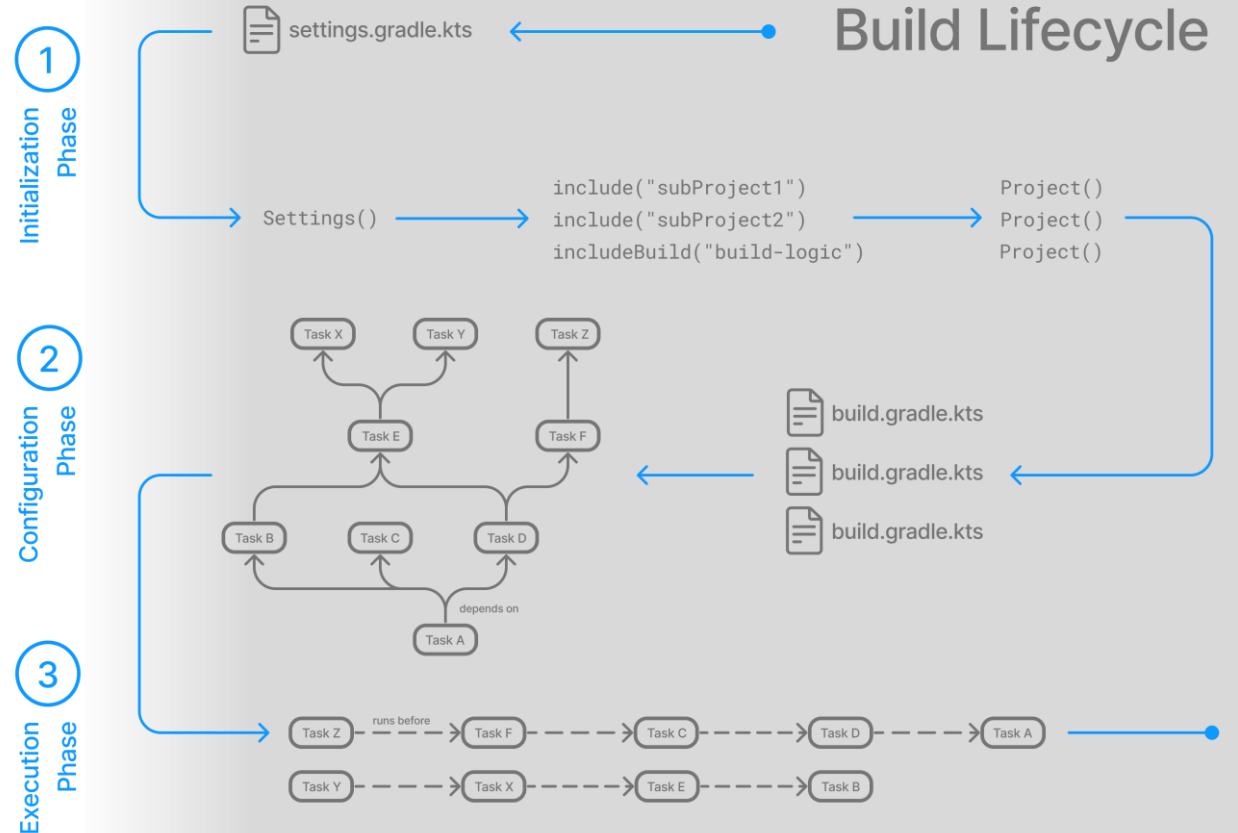
Build Lifecycle

Build Lifecycle

How Gradle organizes and executes builds.

1. Initialization Phase

- Detects root and subprojects via settings.
- Instatiates projects.



Build Lifecycle

How Gradle organizes and executes builds.

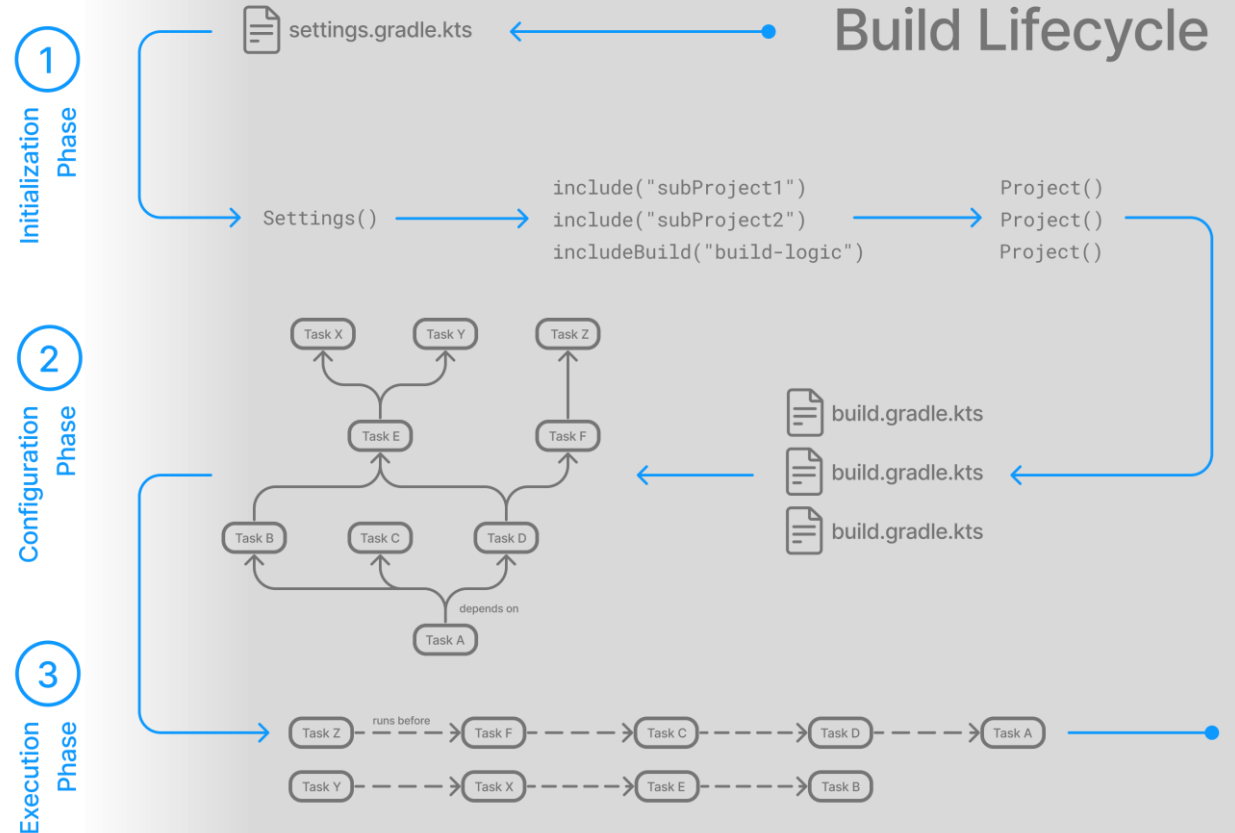
1. Initialization Phase

- Detects root and subprojects via settings.
- Instatiates projects.

2. Configuration Phase

- Configures each project using build files.
- Creates task execution graph.

https://docs.gradle.org/current/userguide/build_lifecycle.html



Build Lifecycle

How Gradle organizes and executes builds.

1. Initialization Phase

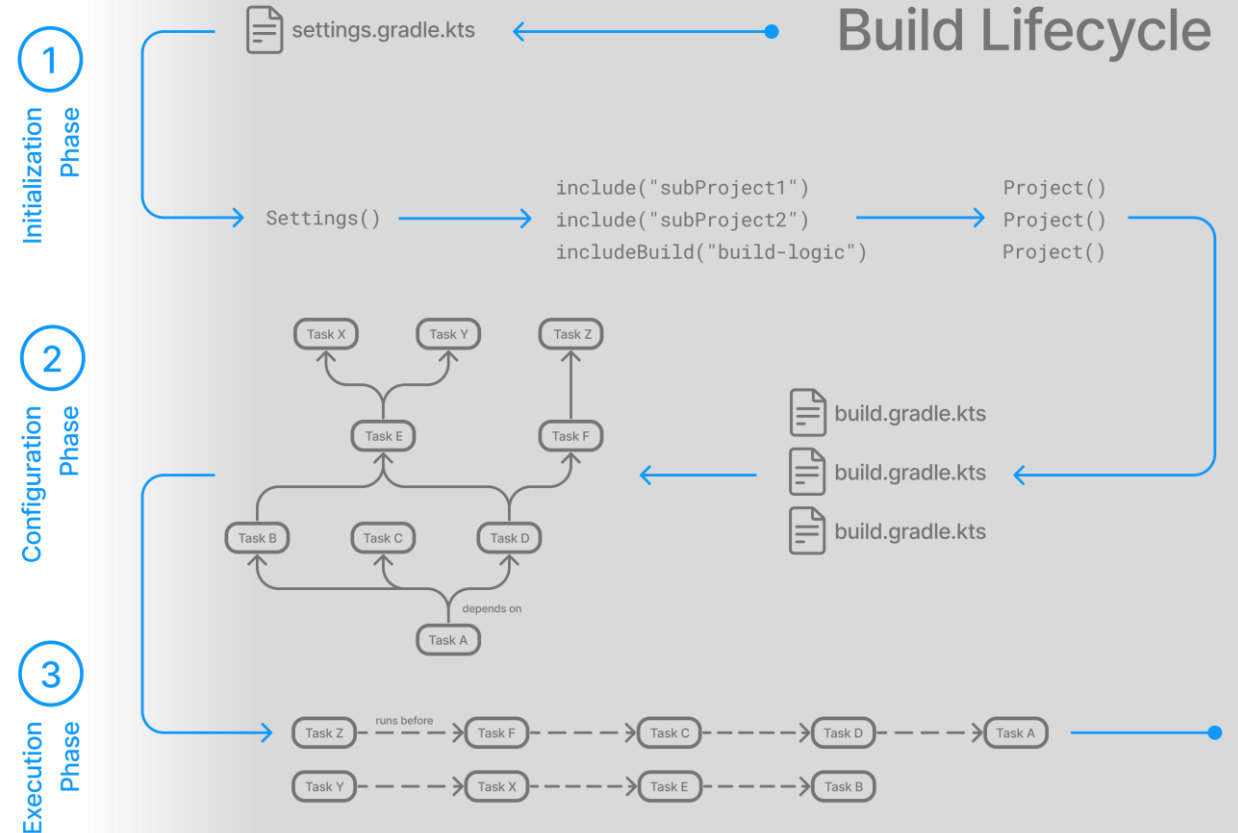
- Detects root and subprojects via settings.
- Instatiates projects.

2. Configuration Phase

- Configures each project using build files.
- Creates task execution graph.

3. Execution Phase

- Runs requested tasks, and dependent ones according to execution graph.



Build Lifecycle

Sample project structure.



https://docs.gradle.org/current/userguide/build_lifecycle.html

Build Lifecycle

Sample task.



```
app/build.gradle.kts

1  println("Hello from :app project config")
2
3  tasks.register("foo") {
4      println("Hello from :app:foo task config")
5      doFirst {
6          println("Hello from :app:foo task run")
7      }
8      doLast {
9          println("Bye from :app:foo task run")
10     }
11     println("Bye from :app:foo task config")
12 }
13
14 println("Bye from :app project config")
```

Build Lifecycle

Project configuration.



```
app/build.gradle.kts

1 println("Hello from :app project config")
2
3 tasks.register("foo") {
4     println("Hello from :app:foo task config")
5     doFirst {
6         println("Hello from :app:foo task run")
7     }
8     doLast {
9         println("Bye from :app:foo task run")
10    }
11    println("Bye from :app:foo task config")
12 }
13
14 println("Bye from :app project config")
```

Build Lifecycle

Project task configuration.



```
1 println("Hello from :app project config")
2
3 tasks.register("foo") {
4     println("Hello from :app:foo task config")
5     doFirst {
6         println("Hello from :app:foo task run")
7     }
8     doLast {
9         println("Bye from :app:foo task run")
10    }
11    println("Bye from :app:foo task config")
12 }
13
14 println("Bye from :app project config")
```

The screenshot shows a code editor window titled 'app/build.gradle.kts'. The code defines a custom task named 'foo' using the 'tasks.register' method. The task has two configuration blocks: 'doFirst' and 'doLast'. The 'doFirst' block prints 'Hello from :app:foo task run' when the task is first executed. The 'doLast' block prints 'Bye from :app:foo task run' when the task is last executed. The task also prints 'Hello from :app project config' at the start and 'Bye from :app:foo task config' at the end of its execution.

Build Lifecycle

Project task execution.

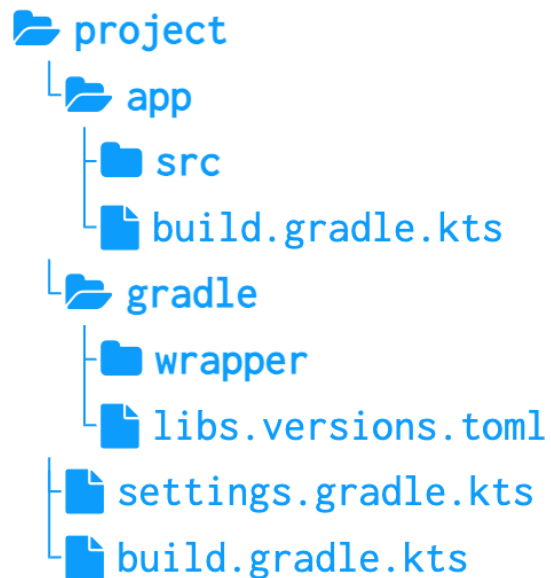


The screenshot shows a code editor window titled 'app/build.gradle.kts'. The code defines a custom task named 'foo' and registers it. The task's configuration block prints a message, and its execution block (doFirst) prints another message. The task is then executed via 'doLast'.

```
1 println("Hello from :app project config")
2
3 tasks.register("foo") {
4     println("Hello from :app:foo task config")
5     doFirst {
6         println("Hello from :app:foo task run")
7     }
8     doLast {
9         println("Bye from :app:foo task run")
10    }
11    println("Bye from :app:foo task config")
12 }
13
14 println("Bye from :app project config")
```

Build Lifecycle

Project task execution output.



```
app/build.gradle.kts

1 println("Hello from :app project config")
2
3 tasks.register("foo") {
4     println("Hello from :app:foo task config")
5     doFirst {
6         println("Hello from :app:foo task run")
7     }
8     doLast {
9         println("Bye from :app:foo task run")
10    }
11    println("Bye from :app:foo task config")
12 }
13
14 println("Bye from :app project config")
```

```
iTerm

~/project $ gradle :app:foo
> Configure project :app
Hello from :app project config
Bye from :app project config
Hello from :app:foo task config
Bye from :app:foo task config

> Task :app:foo
Hello from :app:foo task run
Bye from :app:foo task run

~/project $
```

Build Lifecycle

Project task relation when executing.



```
app/build.gradle.kts

1 println("Hello from :app project config")
2
3 tasks.register("foo") {
4     println("Hello from :app:foo task config")
5     doFirst {
6         println("Hello from :app:foo task run")
7     }
8     doLast {
9         println("Bye from :app:foo task run")
10    }
11    println("Bye from :app:foo task config")
12 }
13
14 tasks.register("bar") {
15     println("Configuration of :app:bar task")
16 }
17
18 println("Bye from :app project config")
```

```
iTerm

~/project $ gradle :app:foo
> Configure project :app
Hello from :app project config
Bye from :app project config
Hello from :app:foo task config
Bye from :app:foo task config

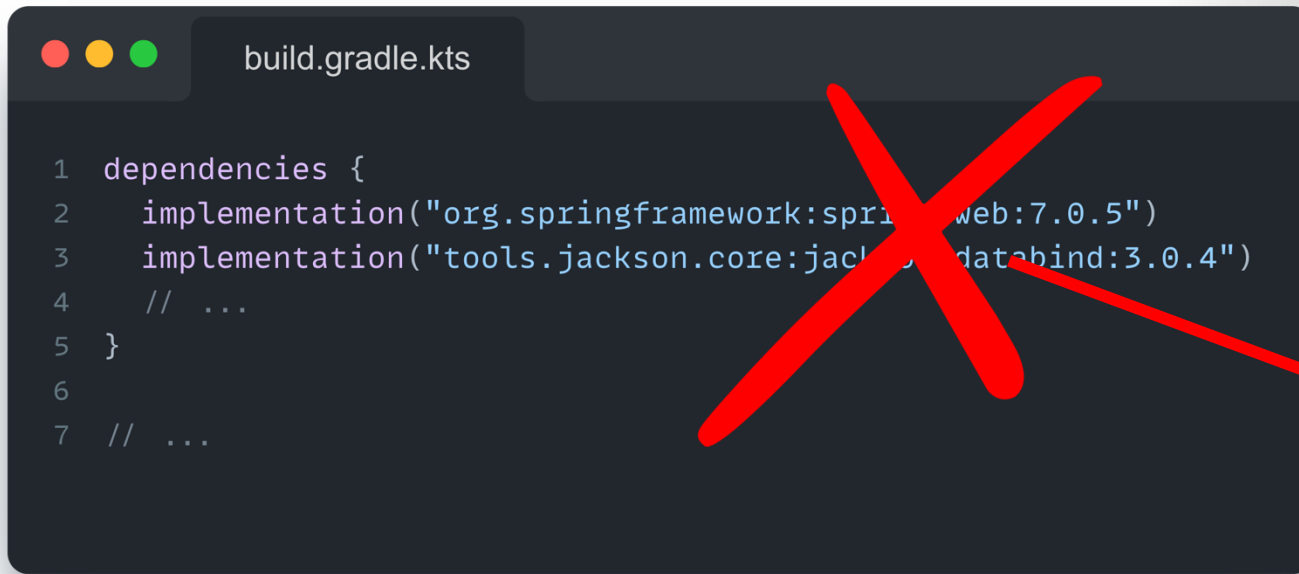
> Task :app:foo
Hello from :app:foo task run
Bye from :app:foo task run

~/project $
```

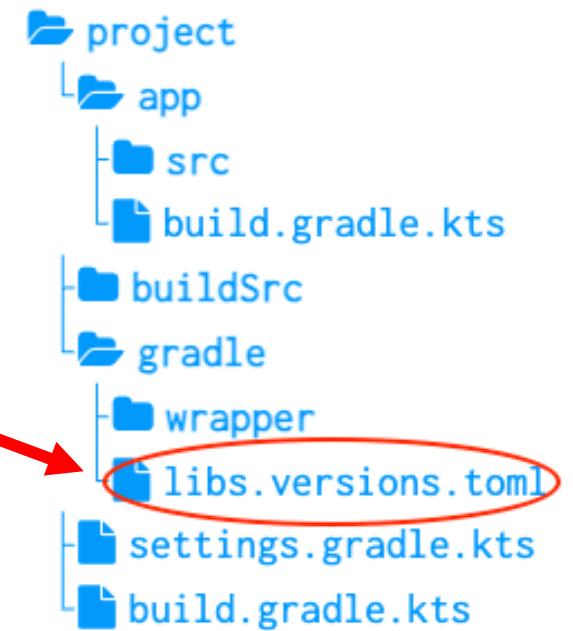



Version Catalogs

Version Catalogs



```
1 dependencies {  
2     implementation("org.springframework:spring-web:7.0.5")  
3     implementation("tools.jackson.core:jackson-databind:3.0.4")  
4     // ...  
5 }  
6  
7 // ...
```



Version Catalogs

Better version management for projects with multiple modules.

```
libs.versions.toml

1 [versions]
2 taskTree = "4.0.1"
3
4 junit = "6.0.3"
5 assertJ = "3.27.7"
6
7 [libraries]
8 spring-web = "org.springframework:spring-web:7.0.5"
9 jackson-databind = "tools.jackson.core:jackson-databind:3.0.4"
10 junit-api = { module = "org.junit.jupiter:junit-jupiter-api", version.ref = "junit" }
11 junit-engine = { module = "org.junit.jupiter:junit-jupiter-engine", version.ref = "junit" }
12 assertj = "org.assertj:assertj-core:3.27.7"
13
14
15 [bundles]
16 testing = [
17     "junit-api",
18     "junit-engine",
19     "assertj"
20 ]
21
22 [plugins]
23 taskTree = { id = "com.dorongold.task-tree", version.ref = "taskTree" }
24
```

```
build.gradle.kts

1 plugins {
2     alias(libs.plugins.taskTree)
3 }
4
5 dependencies {
6     implementation(libs.spring.web)
7     implementation(libs.jackson.databind)
8
9     testImplementation(libs.bundles.testing)
10 }
11
```

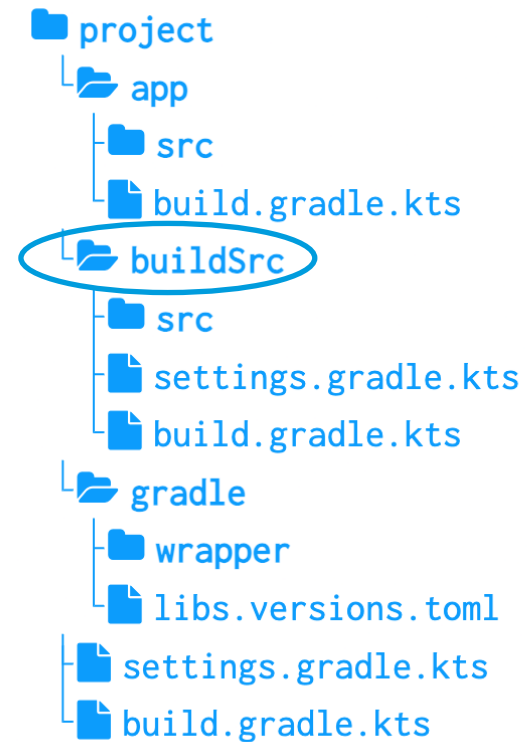


BuildSrc vs Composite Builds

BuildSrc

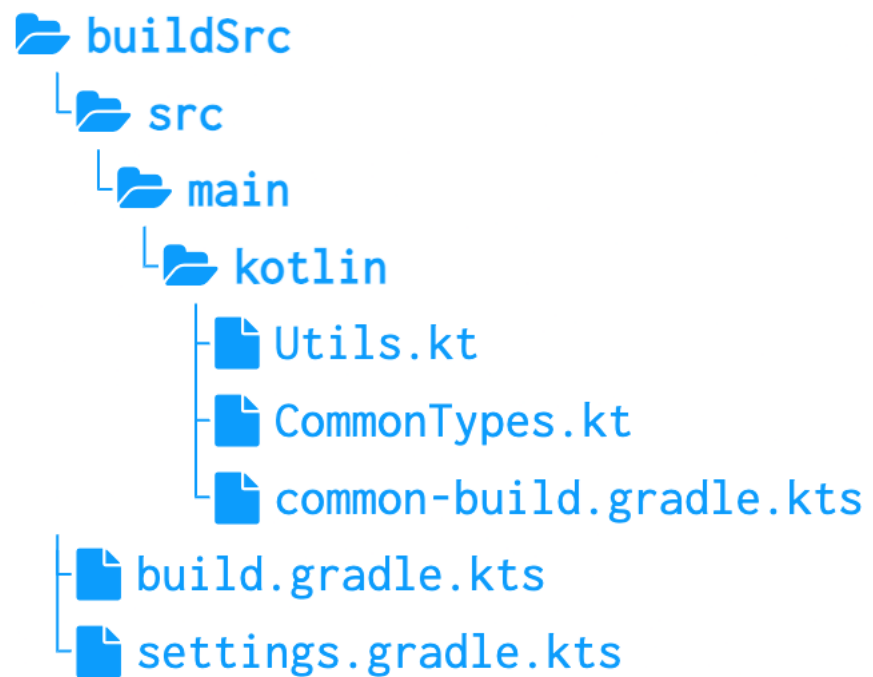
What is this?

- Special Gradle subproject.
 - Only subproject allowed to define a settings file other than the root project.
- Common build logic, tasks, and plugins.
- Compiled during configuration phase before any other project.



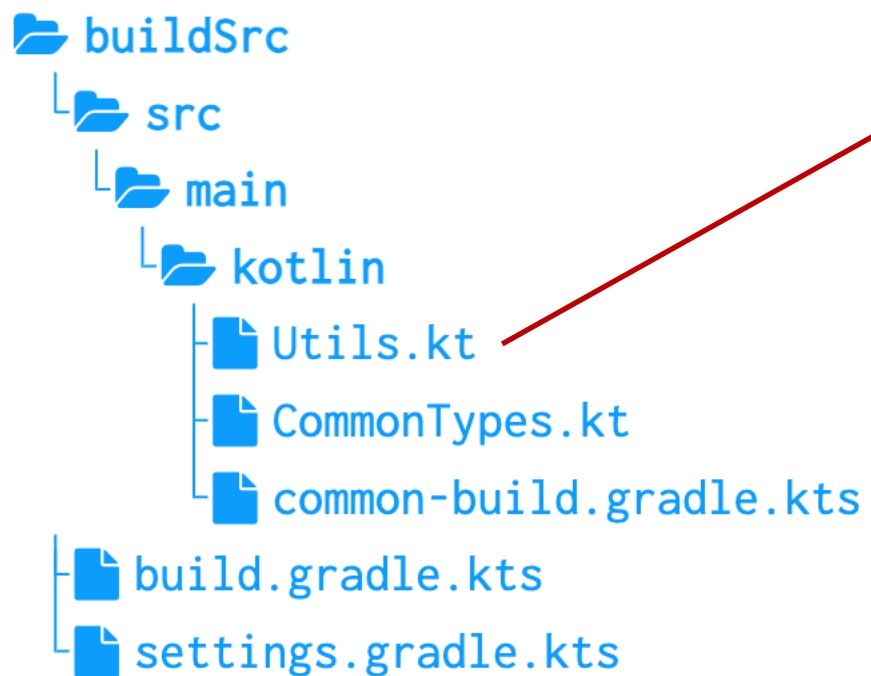
BuildSrc

Usages?



BuildSrc

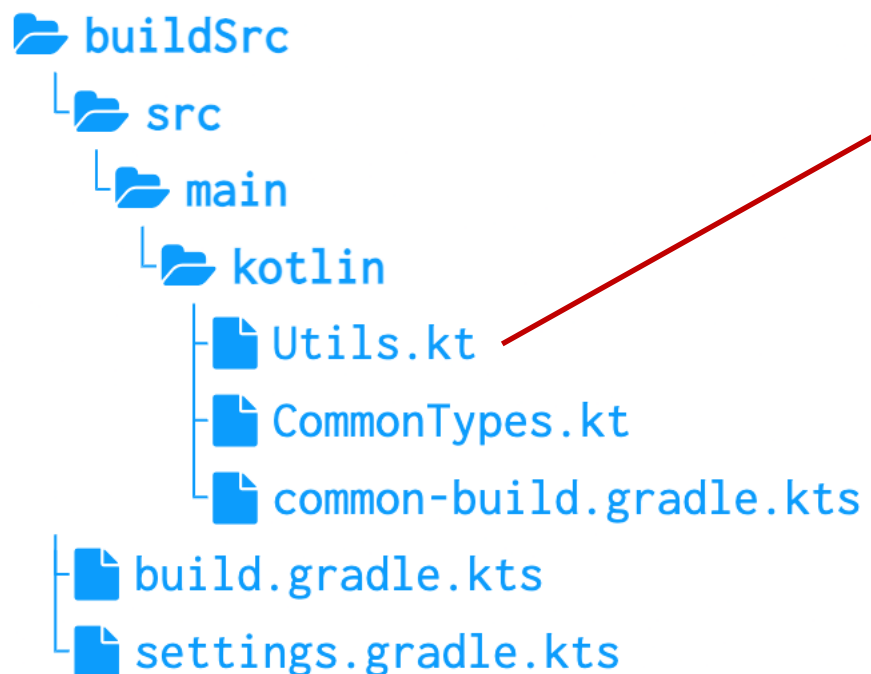
Usages?



- Simple Kotlin files that can be used by all the other subprojects, e.g.:
 - Custom Gradle task definition
 - Common functions
 - Constants

BuildSrc

Usages?



- Simple Kotlin files that can be used by all the other subprojects, e.g.:
 - Custom Gradle task definition
 - Common functions
 - Constants
- Gradle Kotlin Scripting files which will generate pre-compiled Gradle plugins, e.g.:
 - Common tasks
 - Common build configuration

BuildSrc

Pre-compiled plugins.

```
common-kotlin.gradle.kts
1 plugins {
2     kotlin("jvm")
3 }
4
5 repositories {
6     mavenCentral()
7 }
8
9 kotlin {
10     jvmToolchain(libs.versions.java.get().toInt())
11 }
12
13 @Suppress("UnstableApiUsage")
14 testing {
15     suites {
16         getByName<JvmTestSuite>("test") {
17             useJUnitJupiter(libs.versions.jupiter.get())
18         }
19     }
20 }
```

```
buildSrc/build.gradle.kts
1 plugins {
2     `kotlin-dsl`
3 }
4 ...
```



```
sub-project-1/build.gradle.kts
1 plugins {
2     `common-kotlin`
3     // or
4     id("common-kotlin")
5 }
6
7 dependencies { ... }
```

BuildSrc Caveats

Some limitations regarding version catalogs

1. In *buildSrc/build.gradle.kts*

```
buildSrc/settings.gradle.kts

1 dependencyResolutionManagement {
2     // Reuse version catalog from the main build.
3     versionCatalogs {
4         create("libs", { from(files("../gradle/libs.versions.toml")) })
5     }
6 }
7
8 ...
```

```
buildSrc/build.gradle.kts

1 dependencies {
2     implementation(libs.myLib)
3 }
4
5 ...
```

BuildSrc Caveats

Some limitations regarding version catalogs

2. In convention plugins

```
buildSrc/build.gradle.kts

1 dependencies {
2     implementation(files(libs.javaClass.superclass.protectionDomain.codeSource.location))
3     implementation(libs.myLib)
4 }
5
6 ...
```

```
common-kotlin.gradle.kts

1 import org.gradle.accessors.dm.LibrariesForLibs
2
3 val libs = the<LibrariesForLibs>()
4
5 kotlin {
6     jvmToolchain(libs.versions.java.get().toInt())
7 }
8
9 ...
```

BuildSrc Caveats

Some limitations regarding version catalogs

3. When applying plugins in convention plugins

- There is no current direct support 🥲
- You should declare the versions in buildSrc/build.gradle.kts dependencies block 👁️👁️

```
gradle/libs.versions.toml

1 [versions]
2 kotlin = "2.1.10"
3
4 [libraries]
5 kotlinPlugin = { module = "org.jetbrains.kotlin:kotlin-gradle-plugin", version.ref = "kotlin" }
```

```
common-kotlin.gradle.kts

1 plugins {
2     id("org.jetbrains.kotlin.jvm")
3     // or
4     kotlin("jvm")
5 }
6 ...
```

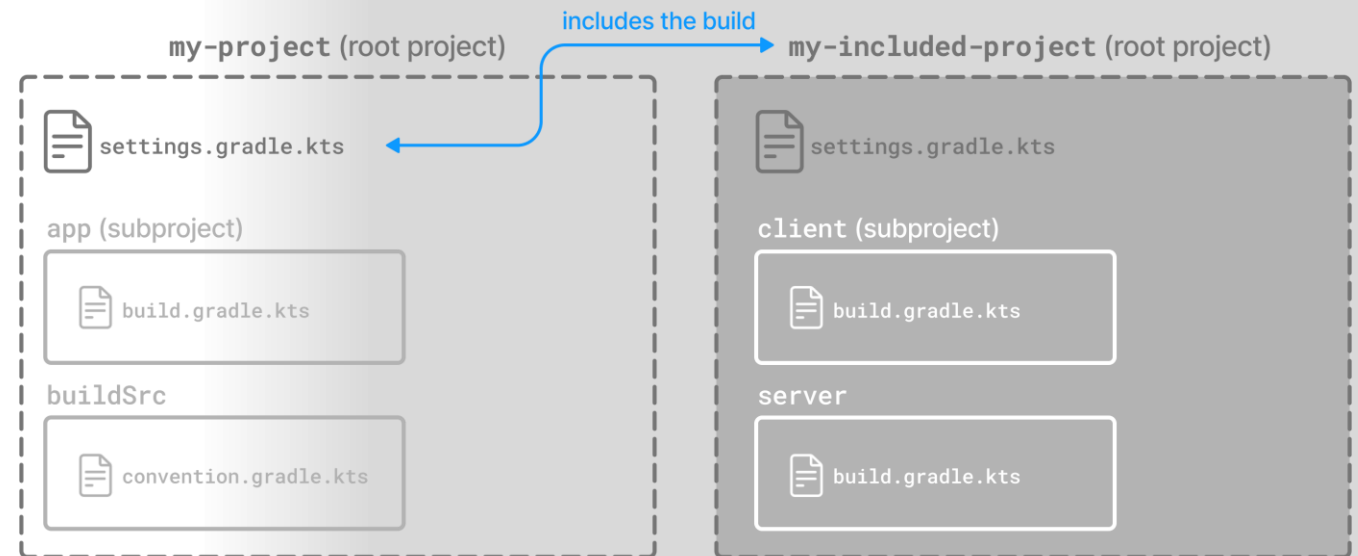
```
buildSrc/build.gradle.kts

1 plugins {
2     `kotlin-dsl`
3 }
4
5 dependencies {
6     ...
7     implementation(libs.kotlinPlugin)
8 }
```

Composite Builds

What are they?

- Gradle's feature to include multiple root projects in the same project.
- Common build logic, tasks, and plugins.
- Almost the same as *buildSrc*, but **explicit**.

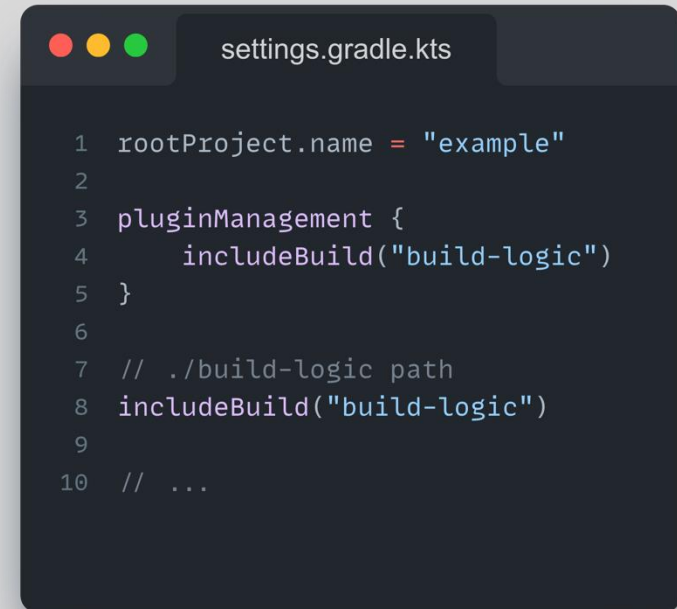


(source: https://docs.gradle.org/current/userguide/composite_builds.html)

Composite Builds

Pros when compared to buildSrc

- Explicit instead of “magically” included submodule.
- Separate builds with own lifecycle instead of single shared one.
- Scales better for large build logic – one change doesn’t force whole recompilation.
- **When including in multiple projects:**
 - Publish the convention plugins instead of doing this ->
 - Use Git submodules because they would need to be in different repositories

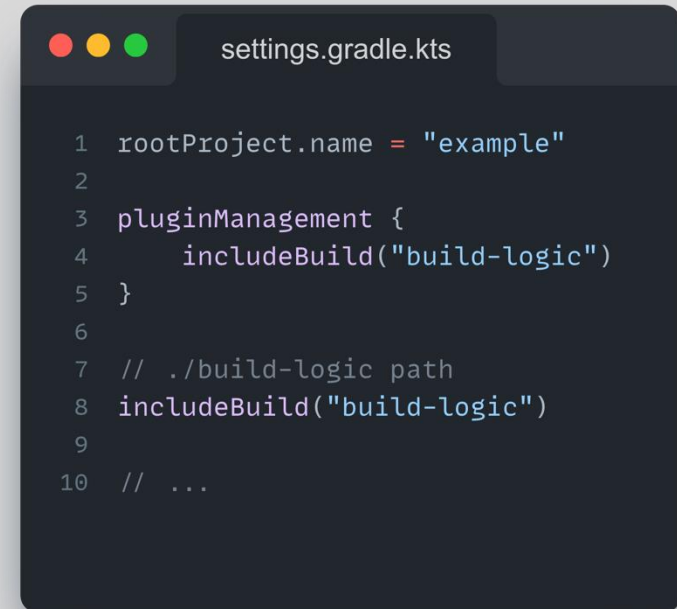
A screenshot of a code editor window titled 'settings.gradle.kts'. The code is written in Kotlin and shows the configuration for a composite build. It includes the root project name, plugin management, and two calls to 'includeBuild' for 'build-logic' at different paths.

```
1 rootProject.name = "example"
2
3 pluginManagement {
4     includeBuild("build-logic")
5 }
6
7 // ./build-logic path
8 includeBuild("build-logic")
9
10 // ...
```

Composite Builds

Cons when compared to buildSrc

- Requires explicit configuration.
- Requires you to use `id("plugin-name")` for convention plugins instead of type-safe properties.
- Might be an overkill for small project setups.
- The same limitations when trying to use version catalogs:
 - In build-logic/build.gradle.kts
 - In convention plugins
 - When applying plugins in convention plugins

A screenshot of a code editor window titled 'settings.gradle.kts'. The code is written in Kotlin and shows the configuration for a composite build. It includes a root project name, a plugin management block with an includeBuild call, and a comment indicating the build logic path.

```
1 rootProject.name = "example"
2
3 pluginManagement {
4     includeBuild("build-logic")
5 }
6
7 // ./build-logic path
8 includeBuild("build-logic")
9
10 // ...
```

BuildSrc vs Composite Builds

Cons when compared to buildSrc

- Use **buildSrc**
 - 👉 Single repo, small logic, and convenience matters.
- Use a **composite build**
 - 👉 Shared logic, growing, or strategic – structure matters.
- You can also leverage build scans (*--scan*) to understand when to switch to composite builds.



Toolchains

Toolchains

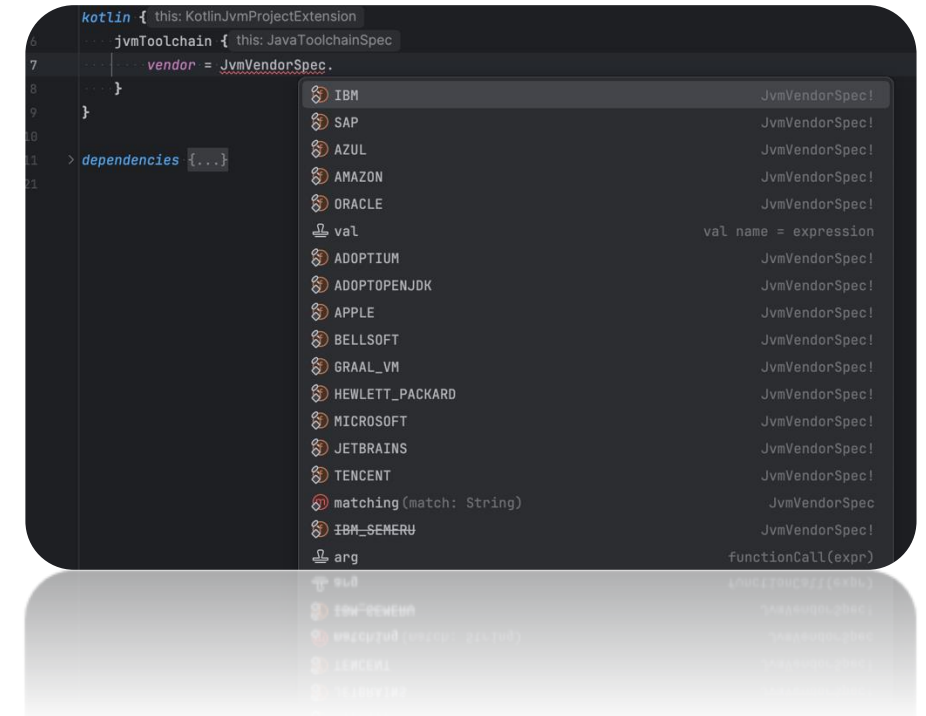
Set of tools to build and run projects (e.g.: local JDK/JRE)

- By default, Gradle uses the same Java toolchain for running Gradle itself and building JVM projects.



```
build.gradle.kts
1 java {
2     toolchain {
3         languageVersion = JavaLanguageVersion.of(21)
4         vendor = JvmVendorSpec.ADOPTIUM
5     }
6 }

build.gradle.kts
1 kotlin {
2     jvmToolchain {
3         languageVersion = JavaLanguageVersion.of(21)
4         vendor = JvmVendorSpec.ADOPTIUM
5     }
6     // or
7     jvmToolchain(21)
8 }
```



Toolchains

Misc.

- When using certain vendor specific toolchains, we might need to configure a Toolchain provider:
 - Manual configuration via *settings.gradle.kts* *toolchainManagement* block.
 - Delegate to a plugin that provides common vendor-specific toolchain repositories (e.g. Foojay).
- We can also control the JVM used by Gradle itself via *updateDaemonJvm* task just like we do with *wrapper* task for Gradle wrapper.

```
settings.gradle.kts

1 plugins {
2     id("org.gradle.toolchains.foojay-resolver-convention")
3         .version("0.9.0")
4 }
5 // or
6 toolchainManagement {
7     jvm {
8         javaRepositories {
9             ...
10        }
11    }
12 }
```

```
build.gradle.kts

1 tasks.named<UpdateDaemonJvm>("updateDaemonJvm") {
2     jvmVersion = JavaLanguageVersion.of(17)
3     jvmVendor = JvmVendorSpec.ADOPTIUM.toString()
4 }
```




Tips & Tricks

Tips & Tricks

- Discovering project tasks with *tasks* task.
- Gradle **TaskTree** plugin – helps with task dependencies.
- Always use *tasks.register* function to create new tasks. 
- Leverage *properties.gradle* and *--parallel* to improve performance.
- Don't use *allprojects* nor *subprojects* blocks, prefer a *buildSrc* approach.
- When using Exec-like tasks in IntelliJ there is an issue with PATH definition for command execution.
- Using *dependencies* task to find actual versions being used by a project.
 - Specify *--configuration <configuration>* to see dependencies for that one only (e.g. *runtimeClasspath*).
 - This is useful when debugging runtime issues such as *NoSuchMethodException*.
- Gradle sometimes provide free training at <https://gradle.com/events>.



Let's

Code



<https://github.com/chriptus13/GradleInit>

Conclusion

- ✅ Gradle Build Lifecycle – Understand phases (Initialization, Configuration, Execution) to optimize builds.
- ✅ BuildSrc & Convention Plugins – Power up your build logic with structured, reusable configurations.
- ✅ Toolchains – Lock Java versions for consistent builds across environments.
- 💡 Key takeaways – Embrace Kotlin DSL for type-safe maintainable builds but beware of hidden caveats!

Q&A





Thank you!
André Martins